However, this paints a naïve picture, because tasks are rarely infinitely divisible in the way that would be required for the equation to hold, and it's also rare for everything to be CPU bound in the way that's assumed. As you've just seen, threads may wait for many things while executing.

One thing that's clear from Amdahl's law is that when you're using concurrency for performance, it's worth looking at the overall design of the application to maximize the potential for concurrency and ensure that there's always useful work for the processors to be doing. If you can reduce the size of the "serial" sections or reduce the potential for threads to wait, you can improve the potential for performance gains on systems with more processors. Alternatively, if you can provide more data for the system to process, and thus keep the parallel sections primed with work, you can reduce the serial fraction and increase the performance gain *P*.

Essentially, scalability is about *reducing the time it takes to perform an action or increasing the amount of data that can be processed in a given time* as more processors are added. Sometimes these are equivalent (you can process more data if each element is processed faster) but not always. Before choosing the techniques to use for dividing work between threads, it's important to identify which of these aspects of scalability are important to you.

I mentioned at the beginning of this section that threads don't always have useful work to do. Sometimes they have to wait for other threads, or for I/O to complete, or for something else. If you give the system something useful to do during this wait, you can effectively "hide" the waiting.

8.4.3. Hiding latency with multiple threads

For lots of the discussions of the performance of multithreaded code, we've been assuming that the threads are running "flat out" and always have useful work to do when they're actually running on a processor. This is of course not true; in application code threads frequently block while waiting for something. For example, they may be waiting for some I/O to complete, waiting to acquire a mutex, waiting for another thread to complete some